

BASTION

Bóveda de credenciales con generador de contraseñas seguras.

Diseño funcional y arquitectónico · fase previa a la codificación

Estudiante David Alexander Benz Zambrano · dabenzza@uide.edu.ec

Carrera Ingeniería en Ciberseguridad · 1er Semestre · Paralelo 1-CIB-1A

Asignatura Lógica de Programación · Docente: Msc. Lilian Aman



Demasiadas claves, mal gestionadas

La proliferación de servicios digitales multiplicó las credenciales que cada persona administra. En la práctica, esto deriva en tres hábitos inseguros que el diseño de BASTION busca corregir.



Reutilización

La misma contraseña en muchos sitios: una filtración compromete todas las cuentas a la vez.



Claves débiles

Contraseñas cortas y predecibles, fáciles de romper por fuerza bruta o diccionario.



Texto plano

Credenciales guardadas en notas o archivos sin cifrar, expuestas ante cualquier acceso.

Las guías de identidad digital del NIST (*Grassi et al., 2017*) advierten que la fortaleza y la unicidad de cada credencial son determinantes para resistir ataques de fuerza bruta y de relleno de credenciales.

¿Qué es BASTION?

Un gestor personal de credenciales que **genera** contraseñas robustas y las **custodia** en una bóveda local cifrada, accesible mediante una sola contraseña maestra.



Contexto profesional: Blue Nova

Nace de una necesidad real de la agencia, que administra a diario accesos a servidores, claves de API y llaves SSH hoy dispersas. Primero como herramienta interna y, en fases posteriores, como base de un servicio para clientes.



Generador seguro

Aleatoriedad criptográfica (RFC 4086).



Bóveda cifrada

AES-256-GCM en reposo.



100% local

Sin nube: menor superficie de ataque.



Por categorías

Organización y búsqueda rápida.

¿Qué buscamos lograr?

OBJETIVO GENERAL Diseñar la solución funcional y arquitectónica de BASTION mediante diagramas de funcionalidad y de arquitectura de software fundamentados en estándares de la industria, en su fase previa a la codificación.

- 1 Investigar y seleccionar los diagramas de funcionalidad y de arquitectura pertinentes para modelar el sistema.
- 2 Analizar el problema e identificar funcionalidades, variables, entradas, salidas y reglas de validación.
- 3 Comparar BASTION con gestores del mercado para delimitar su propuesta de valor y su alcance como MVP.
- 4 Diseñar la arquitectura en capas y el modelo de seguridad, justificando el stack y los estándares aplicados.
- 5 Construir un prototipo funcional de alta fidelidad que valide el diseño y guíe la implementación en Python.

Estándares que sostienen el diseño

Un gestor de contraseñas es, ante todo, un sistema criptográfico: su diseño se apoya en estándares probados, no en mecanismos improvisados.



Aleatoriedad

RFC 4086 — contraseñas impredecibles.



Cifrado

AES-256-GCM — NIST SP 800-38D.



Derivación de clave

Argon2id — resistente a hardware.



Doble factor

TOTP — RFC 6238.



Identidad

NIST SP 800-63B — fortaleza y unicidad.



Modelado

UML 2.5.1 — OMG.

Dónde se posiciona BASTION

Característica	Bitwarden	1Password	LastPass	KeePass	BASTION
Almacenamiento	Nube	Nube	Nube	Local	Local
Código abierto	Sí	No	No	Sí	Sí
Cifrado	AES-256	AES-256	AES-256	AES-256	AES-256-GCM
Costo	Freemium	Pago	Freemium	Gratis	Gratis
Enfoque	General	Equipos	General	Técnico	Técnico / interno

Posicionamiento. BASTION no compite en cobertura de funciones, sino que ocupa un nicho: bóveda local, ligera y auditable, orientada a un perfil técnico que gestiona no solo contraseñas web sino también claves de API y llaves SSH.

¿Qué diagramas investigué y cuáles usé?

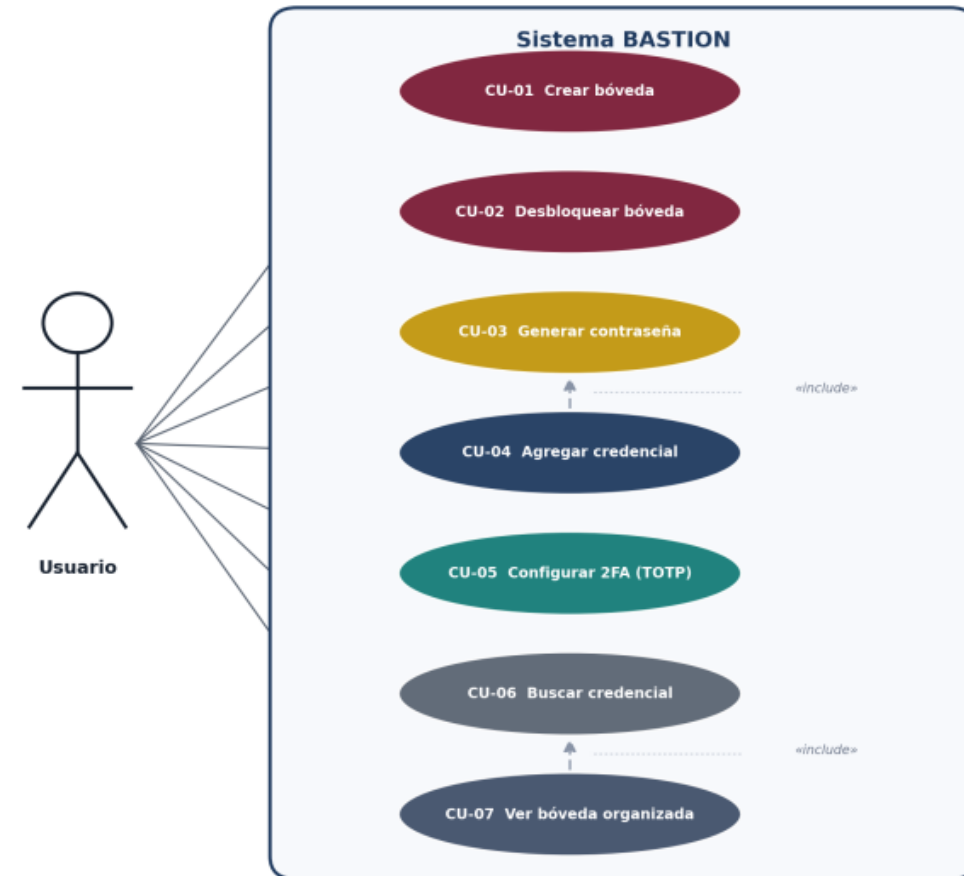
Diagrama	Qué modela	¿Lo uso?	Por qué
Casos de uso	Funciones del sistema y el actor	Sí	Define qué hace y quién lo usa
Diagrama de flujo	Pasos y decisiones del proceso	Sí	Modela la lógica del programa
Actividad	Acciones en paralelo y bifurcaciones	No	Es detalle de implementación
Secuencia	Mensajes entre objetos en el tiempo	No	Es nivel de implementación

Conclusión. Casos de uso y diagrama de flujo se complementan: uno muestra qué hace el sistema y quién lo usa; el otro, cómo ocurre el proceso paso a paso. Los de actividad y secuencia se reservan para la fase de implementación, no para este diseño previo.

Casos de uso

7 casos de uso iniciados por un único actor, el **Usuario**.

Las relaciones «**include**» indican reutilización: agregar una credencial reutiliza la generación de contraseñas, y ver la bóveda reutiliza la búsqueda.



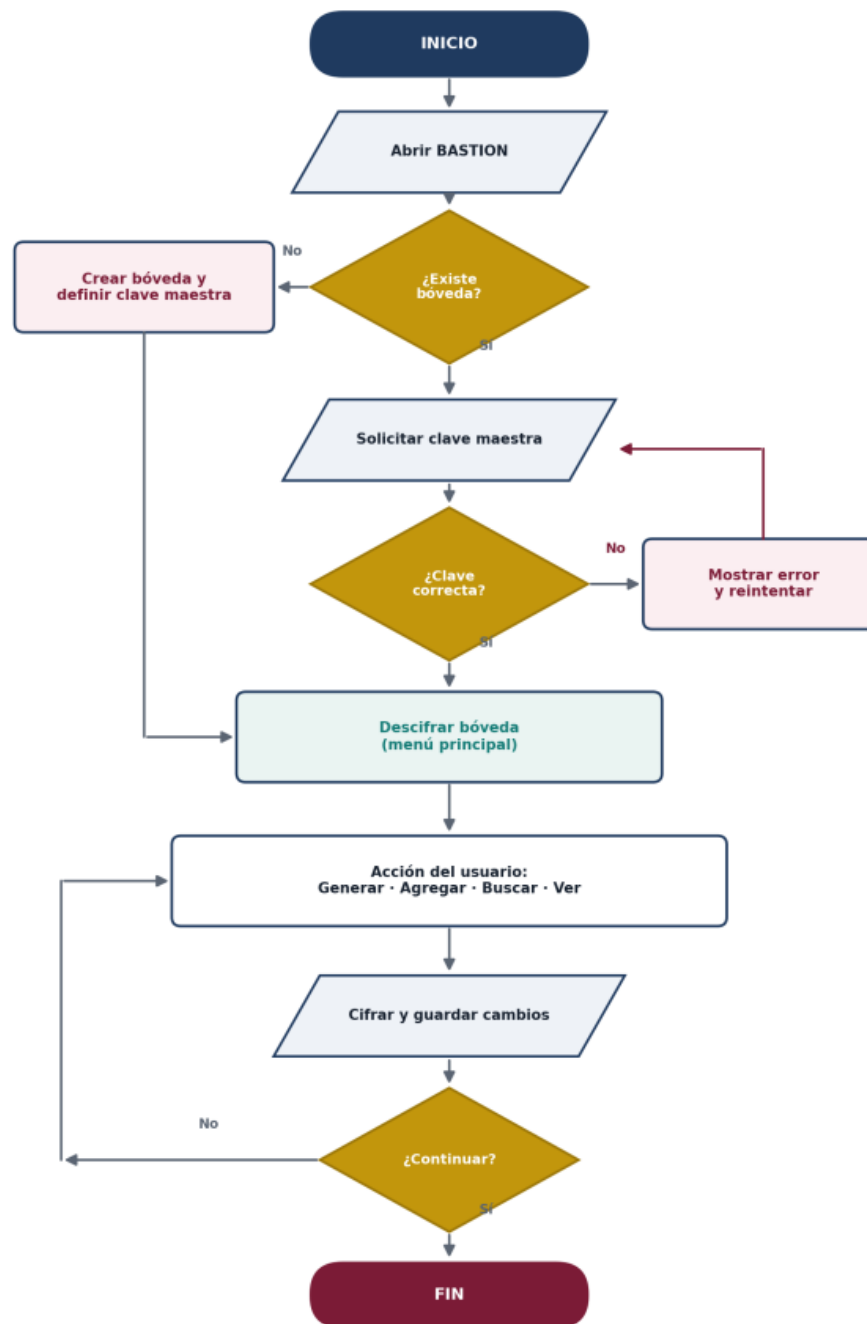
Notación UML 2.5.1 (OMG, 2017)

Flujo del proceso principal

El programa abre, verifica si existe la bóveda y autentica con la contraseña maestra. Si es correcta, descifra y entra al menú; si no, reintenta.

Desde ahí, el usuario genera, agrega, busca o consulta credenciales; cada cambio se cifra y se guarda antes de decidir si continúa.

Norma ISO 5807



¿Qué arquitectura investigué y cuál usé?

Estilo	Idea central	¿Lo uso?	Por qué
En capas	Presentación, lógica y datos separados	Sí	Separa interfaz, reglas y cifrado
Monolítico	Todo el código en una sola unidad	No	Difícil de mantener y escalar
MVC	Modelo, vista y controlador	No	Está pensado para apps web
Cliente-servidor	Un servidor central y varios clientes	No	BASTION es local, sin servidor
Microservicios	Servicios independientes y desplegados	No	Es excesivo para una app local

Conclusión. La arquitectura en tres capas da la separación más clara entre interfaz, lógica de negocio y almacenamiento cifrado para una aplicación local. Los demás estilos añaden complejidad que no aporta en esta fase del proyecto.

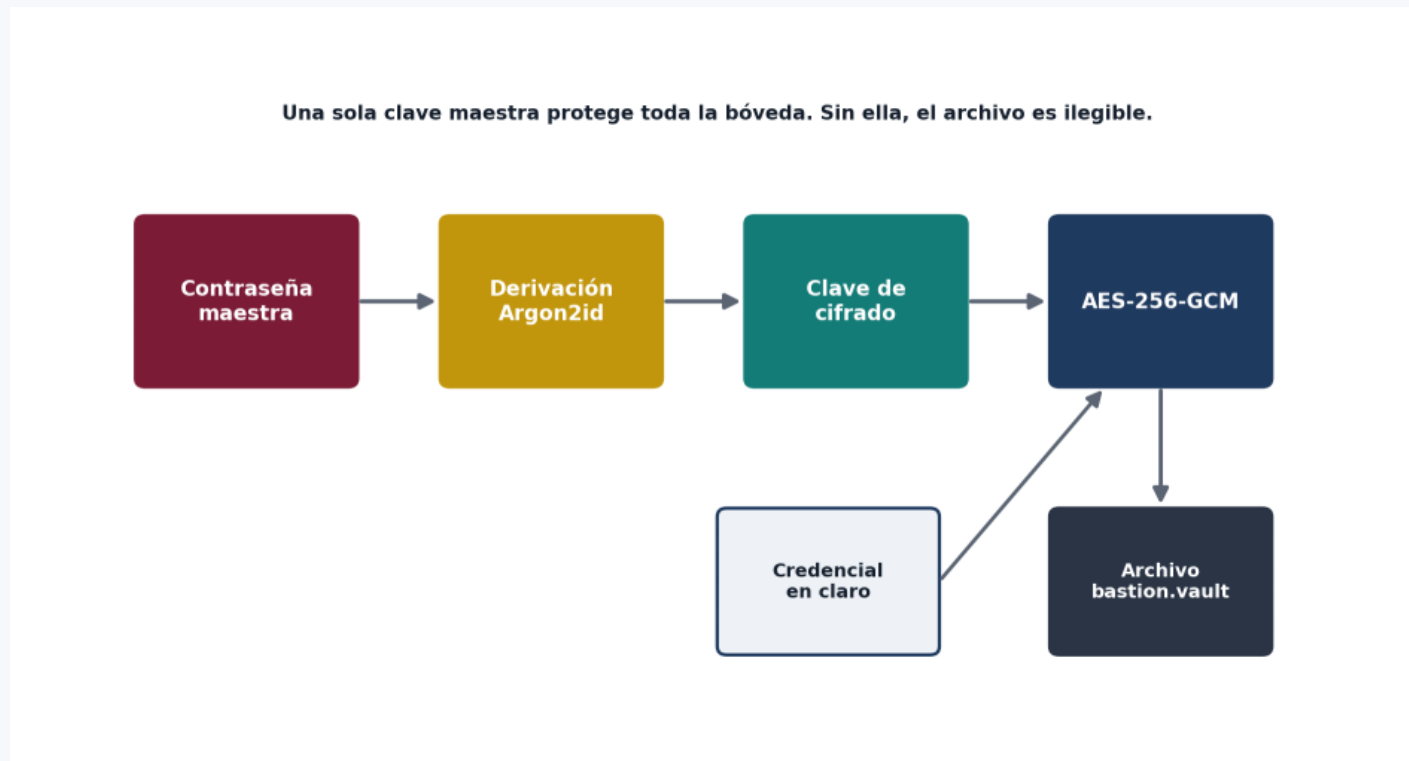
Arquitectura en tres capas

Presentación (interfaz), **Lógica de negocio** (reglas del dominio) y **Persistencia** (almacenamiento cifrado). La separación permite cambiar la interfaz sin alterar la lógica ni el almacenamiento.



Modelo de cifrado

La contraseña maestra no se usa directamente: **Argon2id** la transforma en una clave robusta con la que **AES-256-GCM** cifra cada credencial. Sin la clave maestra, el archivo permanece ilegible.



Cómo se construye con Python

El MVP se programa en **Python 3**, apoyado en bibliotecas de seguridad maduras en lugar de reinventar la criptografía.

BIBLIOTECAS CLAVE

secrets aleatoriedad criptográfica

string conjuntos de caracteres

cryptography cifrado AES-256-GCM

argon2-cffi derivación Argon2id

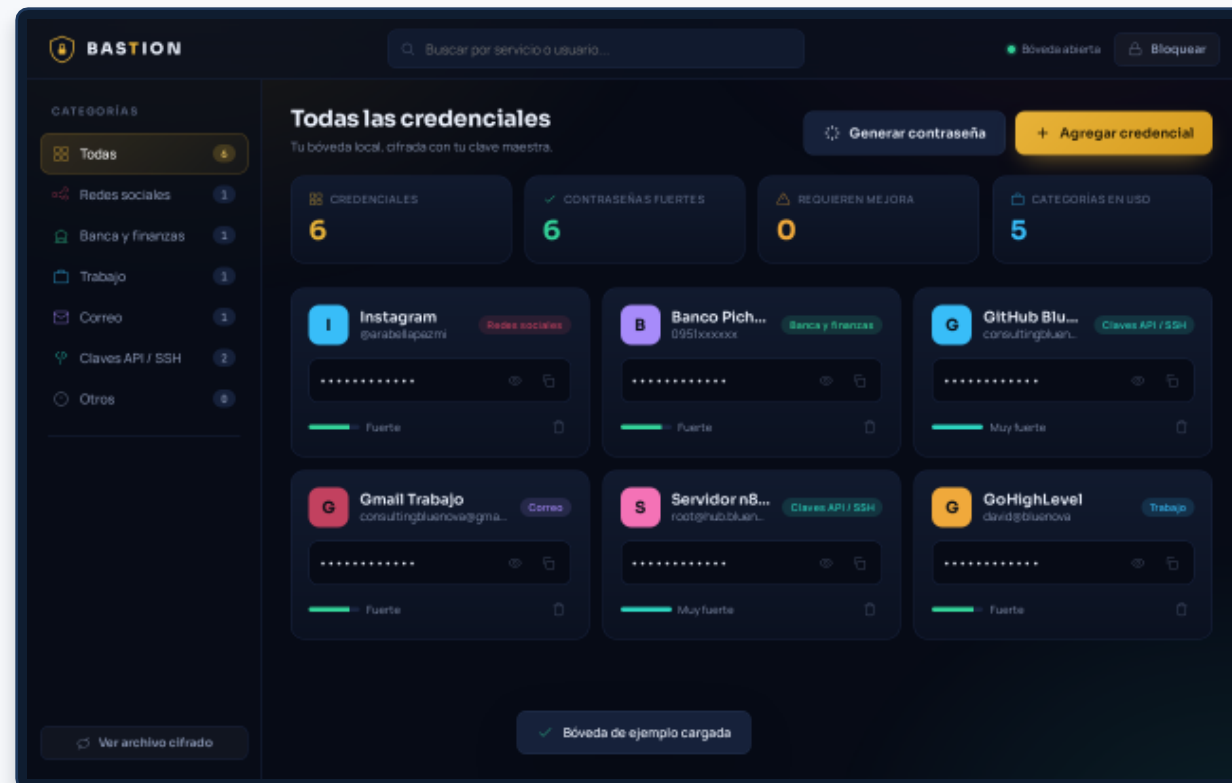
json guarda las credenciales

getpass lee la clave sin mostrarla

```
semana_3_LDP.py U •
modulo-2 > semana_3_LDP.py > ...
1  import secrets
2  import string
3
4  def generar_contrasena(longitud, simbolos):
5      abc = string.ascii_letters + string.digits
6      if simbolos:
7          abc += "!@#$%&*?-_ "
8      clave = ""
9      for i in range(longitud):
10         clave += secrets.choice(abc)
11     return clave
12
13     clave = generar_contrasena(16, True)
14
15     print(clave)
```

Interfaz de alta fidelidad

Para validar el diseño se construyó un prototipo funcional y navegable: crear la bóveda, generar contraseñas, agregar credenciales y consultar el archivo cifrado. Es la maqueta que guiará la implementación en Python.



¿Hasta dónde llega BASTION?

